

Lab Activity: Introduction to Bash Scripting

Environment Setup You can complete this lab using a native Linux Terminal (or WSL on Windows) OR using Google Colab.

- **If using Linux:** Open your terminal, use a text editor (like `nano` or `gedit`) to write your scripts, save them with a `.sh` extension, and run them using `bash filename.sh`.
- **If using Google Colab:** Open a new Notebook. To run Bash commands instead of Python, place `%%bash` at the very top of your code cell.

Activity 1: Basic Output and Redirection (> and >>) In Bash, we use `echo` to print text. We can route that text into files using redirection operators.

- `>` overwrites a file completely.
- `>>` appends to the end of a file.

The Code:

```
%%bash
# Line 1: Creates a new file (or overwrites it) with the text "Log
started."
echo "Log started." > system_log.txt

# Line 2: Appends the text "Operation successful." to the next line
of the file.
echo "Operation successful." >> system_log.txt

# Line 3: Prints the contents of the file to the screen to verify.
cat system_log.txt
```

Activity 2: Variable Catching \$(), Pipes |, and grep We can save the output of a command into a variable using \$(). We can also connect commands together using a pipe |, which takes the output of the first command and feeds it directly into the next. **grep** is a tool used to search for specific text.

The Code:

```
%%bash
# Line 1: Runs the 'date' command and stores the result inside the
variable 'current_time'.
current_time=$(date)

# Line 2: Prints the stored variable to the terminal.
echo "The script was run on: $current_time"

# Line 3: Lists all files (ls -l), then pipes (|) that list into
grep, which filters and shows ONLY lines containing ".txt".
ls -l | grep ".txt"
```

Activity 3: Loops and the Black Hole (/dev/null) Loops allow us to repeat actions. If a command produces errors or output we don't want to see, we can throw it away by redirecting it to /dev/null, which is essentially a digital black hole.

The Code:

```
%%bash
```

```
# Line 1: Starts a 'for' loop that will run exactly 3 times,  
assigning 1, 2, and 3 to variable 'i'.
```

```
for i in 1 2 3
```

```
do
```

```
# Line 2: Prints the current loop number.
```

```
echo "Processing item $i..."
```

```
# Line 3: Tries to list a fake directory. We redirect the error  
output (2>) to /dev/null so it hides the error from the screen.
```

```
ls /this_does_not_exist 2> /dev/null
```

```
done
```

```
# Line 4: A simple while loop. Sets a counter.
```

```
counter=1
```

```
# Line 5: Loops as long as the counter is less than or equal to 2 (-  
le 2).
```

```
while [ $counter -le 2 ]
```

```
do
```

```
echo "While loop count: $counter"
```

```
# Line 6: Adds 1 to the counter to prevent an infinite loop.
```

```
((counter++))
```

```
done
```